CODING CHALLENGE

1. K-Nearest Neighbour (KNN)

```
import math
from collections import Counter

# ------------------------------------------------
# K-Nearest Neighbour (KNN)
# ------------------------------------------------

# Training dataset
# Features: [Height, Weight]
X_train = [
    [5.0, 50],
    [5.2, 55],
    [5.5, 60],
    [5.8, 65],
    [6.0, 70],
    [6.2, 75],
    [6.5, 80],
    [6.7, 85]
]

# Class labels
y_train = [
    "Class A",
    "Class A",
    "Class A",
    "Class A",
    "Class B",
    "Class B",
    "Class B",
    "Class B"
]

# Test dataset
X_test = [
    [5.4, 58],
    [6.3, 78],
    [5.7, 63],
    [6.6, 83]
]

y_test = [
    "Class A",
    "Class B",
    "Class A",
    "Class B"
]


# Euclidean distance function
def euclidean_distance(point1, point2):
    distance = 0

    for i in range(len(point1)):
        distance += (point1[i] - point2[i]) ** 2

    return math.sqrt(distance)


# KNN prediction function
def knn_predict(X_train, y_train, test_point, k):

    distances = []

    # Calculate distance from test point to all training points
    for i in range(len(X_train)):
        distance = euclidean_distance(test_point, X_train[i])
        distances.append((distance, y_train[i]))

    # Sort according to distance
    distances.sort(key=lambda x: x[0])
```

```
    # Select K nearest neighbours
    nearest_neighbors = distances[:k]

    # Get their class labels
    labels = [label for distance, label in nearest_neighbors]

    # Majority voting
    prediction = Counter(labels).most_common(1)[0][0]

    return prediction


# User input for K
k = int(input("Enter the value of K: "))

if k <= 0 or k > len(X_train):
    print("Invalid value of K")
else:

    predictions = []

    print("\nPredictions:")
    print("---------------------------")

    for i in range(len(X_test)):

        prediction = knn_predict(
            X_train,
            y_train,
            X_test[i],
            k
        )

        predictions.append(prediction)

        print(
            "Test Instance:", X_test[i],
            "Actual:", y_test[i],
            "Predicted:", prediction
        )

    # Calculate accuracy
    correct = 0

    for i in range(len(y_test)):
        if predictions[i] == y_test[i]:
            correct += 1

    accuracy = (correct / len(y_test)) * 100

    print("---------------------------")
    print("Accuracy:", accuracy, "%")

    # Predict a new instance
    print("\nEnter values for a new test instance:")
    height = float(input("Height: "))
    weight = float(input("Weight: "))

    new_point = [height, weight]

    new_prediction = knn_predict(
        X_train,
        y_train,
        new_point,
        k
    )

    print("Predicted Class:", new_prediction)
```

```
Enter the value of K: 3

Predictions:
---------------------------
Test Instance: [5.4, 58] Actual: Class A Predicted: Class A
Test Instance: [6.3, 78] Actual: Class B Predicted: Class B
Test Instance: [5.7, 63] Actual: Class A Predicted: Class A
```

```
Test Instance: [6.6, 83] Actual: Class B Predicted: Class B
--------------------------
Accuracy: 100.0 %

Enter values for a new test instance:
Height: 34
Weight: 56
Predicted Class: Class A
```

2. Logistic Regression Using Gradient Descent

```
import numpy as np

# ---------------------------------------------------
# Logistic Regression using Gradient Descent
# ---------------------------------------------------

# Training data
# Feature 1 = Hours Studied
# Feature 2 = Attendance (%)

X_train = np.array([
    [1, 40],
    [2, 45],
    [2, 50],
    [3, 55],
    [3, 60],
    [4, 65],
    [4, 70],
    [5, 75],
    [5, 80],
    [6, 85]
], dtype=float)

# Target
# 0 = Fail
# 1 = Pass

y_train = np.array([
    0, 0, 0, 0, 0,
    1, 1, 1, 1, 1
], dtype=float)


# Test data
X_test = np.array([
    [2, 48],
    [3, 58],
    [4, 68],
    [5, 78],
    [6, 88]
], dtype=float)

y_test = np.array([0, 0, 1, 1, 1])


# ---------------------------------------------------
# Feature scaling
# ---------------------------------------------------

mean = X_train.mean(axis=0)
std = X_train.std(axis=0)

X_train_scaled = (X_train - mean) / std
X_test_scaled = (X_test - mean) / std


# ---------------------------------------------------
# Sigmoid function
# ---------------------------------------------------

def sigmoid(z):
    return 1 / (1 + np.exp(-z))


# ---------------------------------------------------
```

```
# Logistic Regression
# ------------------------------------------------

def train_logistic_regression(X, y, learning_rate, epochs):

    # Initialize weights and bias
    weights = np.zeros(X.shape[1])
    bias = 0

    for epoch in range(epochs):

        # Linear equation
        z = np.dot(X, weights) + bias

        # Sigmoid output
        predictions = sigmoid(z)

        # Calculate errors
        error = predictions - y

        # Calculate gradients
        dw = np.dot(X.T, error) / len(y)
        db = np.sum(error) / len(y)

        # Update weights
        weights = weights - learning_rate * dw

        # Update bias
        bias = bias - learning_rate * db

    return weights, bias


# ------------------------------------------------
# Prediction function
# ------------------------------------------------

def predict(X, weights, bias):

    z = np.dot(X, weights) + bias

    probabilities = sigmoid(z)

    # Convert probability to class
    predictions = (probabilities >= 0.5).astype(int)

    return probabilities, predictions


# ------------------------------------------------
# User input for learning rate
# ------------------------------------------------

learning_rate = float(
    input("Enter learning rate (example: 0.1): ")
)

epochs = 1000


# Train model
weights, bias = train_logistic_regression(
    X_train_scaled,
    y_train,
    learning_rate,
    epochs
)


# Predict test data
probabilities, predictions = predict(
    X_test_scaled,
    weights,
    bias
)
```

```
# ------------------------------------------------
# Display results
# ------------------------------------------------

print("\nLogistic Regression Results")
print("-----------------------------------")

for i in range(len(X_test)):

    print(
        "Test Instance:", X_test[i],
        "Actual:", int(y_test[i]),
        "Sigmoid Output:", round(probabilities[i], 4),
        "Predicted:", predictions[i]
    )


# Calculate accuracy
accuracy = np.mean(predictions == y_test) * 100

print("-----------------------------------")
print("Weights:", weights)
print("Bias:", bias)
print("Accuracy:", round(accuracy, 2), "%")
```

```
Enter learning rate (example: 0.1): 07

Logistic Regression Results
-----------------------------------
Test Instance: [ 2. 48.] Actual: 0 Sigmoid Output: 0.0 Predicted: 0
Test Instance: [ 3. 58.] Actual: 0 Sigmoid Output: 0.0013 Predicted: 0
Test Instance: [ 4. 68.] Actual: 1 Sigmoid Output: 0.9993 Predicted: 1
Test Instance: [ 5. 78.] Actual: 1 Sigmoid Output: 1.0 Predicted: 1
Test Instance: [ 6. 88.] Actual: 1 Sigmoid Output: 1.0 Predicted: 1
-----------------------------------
Weights: [11.6681995  8.7068225]
Bias: 3.4629173443698204e-15
Accuracy: 100.0 %
```